

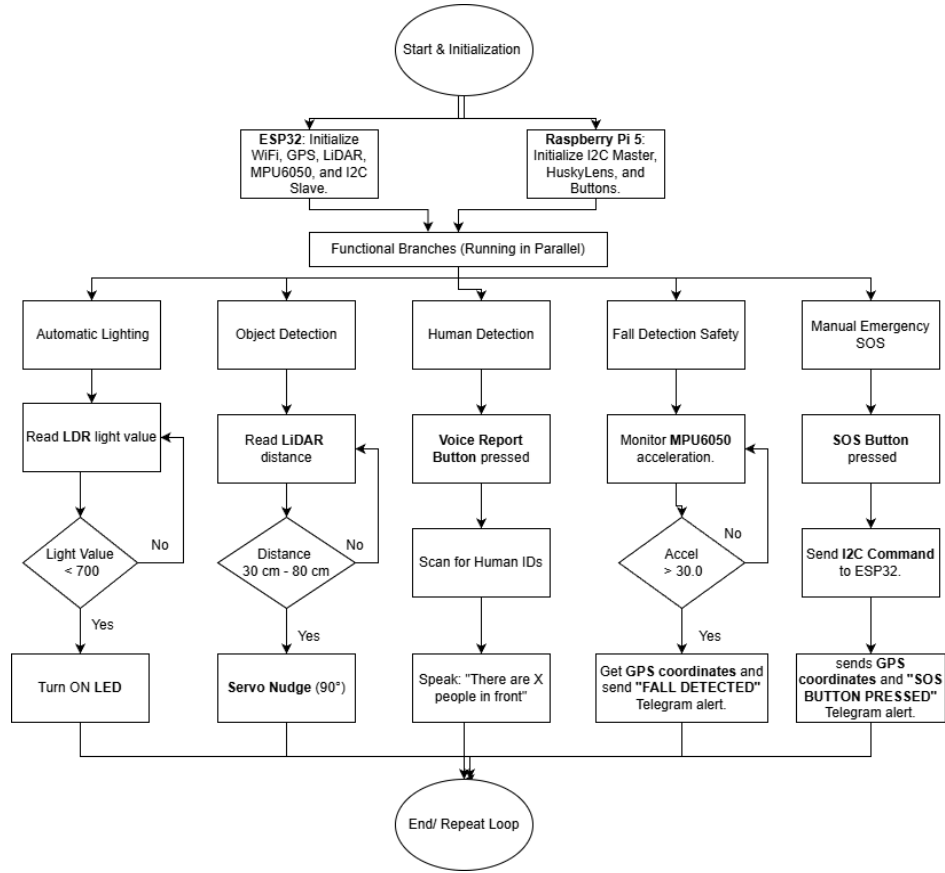


Fig. 3 Flowchart explaining the work principle of IntelliCane

2.2 Schematic Diagram and Prototype Setup

Fig. 4 is the prototype design of the IntelliCane which highlights the mechanical setup and component placement. It illustrates the mounting of the components, alongside the actual internal arrangement of the Raspberry Pi 5 and ESP32 within the custom enclosure. Additionally, Fig. 5 exhibits the complete circuit diagram of the system, detailing the wiring connections between the power management unit, the dual-processor architecture, and the peripheral sensors for System 1 (Automated Lighting), System 2 (Emergency Alert), System 3 (Object Detection), and System 4 (Human Detection).

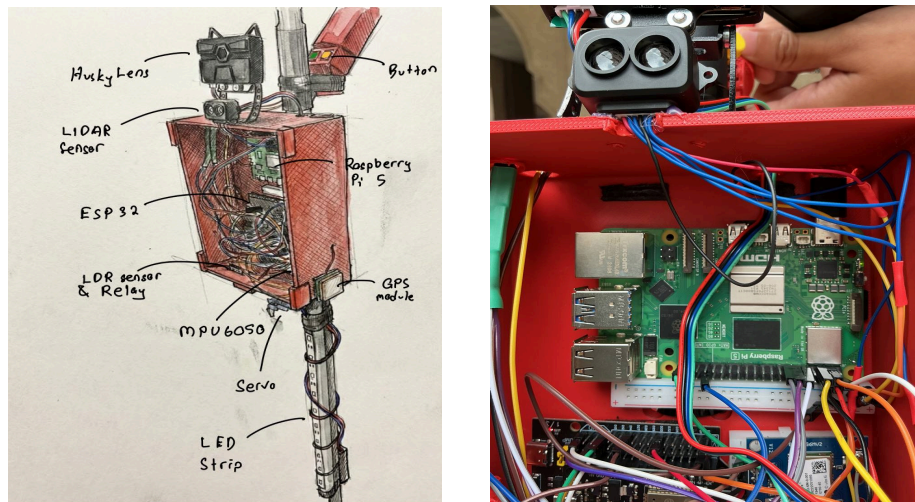


Fig. 4 Prototype Schematics and the actual setup

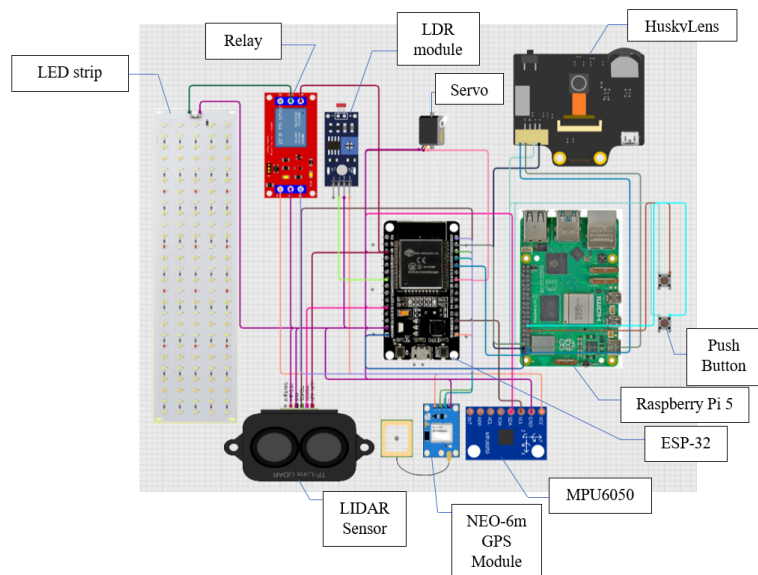


Fig. 5 Circuit Schematics on Cirkuit Designer

3. RESULT AND DISCUSSION

3.1 System 1 (Automated Lighting System) Analysis

The lighting system, designated as System 1, was designed to solve the visibility issue for users walking in low-light environments. The hardware setup consists of an LDR sensor connected to the ESP32 and an LED strip controlled via a Relay Module. The main goal of this test was to verify that the relay triggers reliably at the correct light threshold without flickering.

In the firmware, we set a specific threshold value of 700 for the analog reading. The logic is simple: if the analog value drops below 700, the system assumes it is "dark" and pulls the relay signal LOW to close the circuit and turn on the LED strip. If the value is above 700, the relay stays HIGH (open), keeping the light off to save battery. We tested this by simulating day and night cycles. For the "bright" condition, the cane was placed under normal room lighting. For the "dark" condition, we covered the LDR sensor completely to block out ambient light. As seen in **Fig. 6**, the system responded immediately to the changes. The relay audibly clicked and the LEDs turned on as soon as the sensor was covered, confirming the threshold of 700 is effective for distinguishing between a lit room and darkness. Table 2 below summarizes the behavior we observed during the functional test.

Table 2: System 1 Response Test

Test Condition	Analog Reading (Approx.)	Relay State	LED Strip Status
Bright / Ambient Light	> 700	HIGH (Open)	OFF
Dark	< 700	LOW (Closed)	ON



a) *Bright*



b) *Dark*

Fig. 6 Automatic light system for two condition, bright and dark

3.2 System 2 (Emergency Alert System) Analysis

The Emergency Alert System, designated as System 2, acts as the primary safety net for the user. It triggers under two conditions: the first is manual, where the user presses the SOS button during a panic situation; the second is automatic, where the cane detects a sudden fall. In both scenarios, the main goal is to send the user's precise GPS location to a guardian via Telegram. We implemented the logic on the ESP32 to handle the communication. The fall detection algorithm uses the MPU6050 accelerometer. In the firmware, we calculate the total acceleration vector using the square root formula. Through trial and error by dropping the cane on a mattress, we set a trigger threshold of 30.0 (approximately 3G). We also added a 5-second cooldown timer in the code to prevent the system from sending repeated alerts if the user picks the cane up immediately after a fall.

For the manual SOS, the button is connected to the Raspberry Pi. When pressed, the Pi sends an I2C command (Value: 1) to the ESP32. The ESP32 then fetches the latest coordinates from the NEO-6M GPS module. We observed that the GPS accuracy relies heavily on the number of satellites acquired. The system works significantly better in open spaces where the satellite signal is strong, achieving an accuracy of approximately 5 meters. In indoor environments, the location fix may take longer or be less precise.

Fig. 7 shows the actual alerts received on the guardian's phone during testing. The first message (12:15 PM) shows the response to the manual button press and identifies the reason as "SOS BUTTON PRESSED". The second message (12:16 PM) was triggered by a simulated fall and identifies the reason as "FALL DETECTED". In both cases, the system successfully generated a clickable Google Maps URL which directs the guardian to the exact location. We also measured the latency, which is the time between the event and the message arriving on the phone. As shown in Table 3, the manual button is slightly faster because it does not require the calculation and verification time needed for the fall detection algorithm

Table 3: Emergency System Response Latency

Trigger Type	Sensor Source	Average Latency	GPS Condition
Manual SOS	Physical Button (via I2C)	~ 2 - 3 seconds	Best in Open Space
Fall Detection	MPU6050 (Acceleration > 30.0)	~ 4 - 5 seconds	Best in Open Space

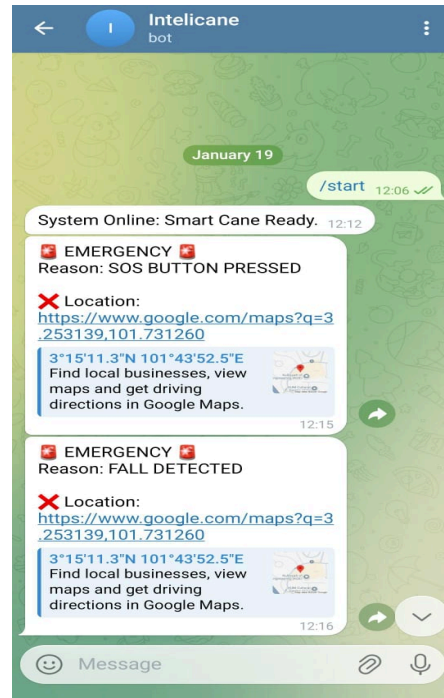


Fig. 7 Emergency message received by two condition via Telegram

3.3 System 3 (Object Detection System) Analysis

System 3 handles obstacle detection using a TF-Luna LiDAR sensor and a micro-servo motor. The goal of this system is to provide discreet haptic feedback to the user when an obstacle is within a critical range, without relying on audio alerts that might be drowned out in noisy environments.

The firmware logic polls the LiDAR sensor every 100 milliseconds. This sampling rate was chosen to ensure real-time responsiveness while preventing the servo from jittering due to sensor noise. We defined a specific "Safety Zone" in the code between 30 cm and 80 cm, as illustrated in Fig. 8. Any object detected closer than 30 cm is ignored to prevent false alarms caused by the user's own feet or the ground during the swing of the cane. Objects further than 80 cm are considered safe and do not trigger an alert.

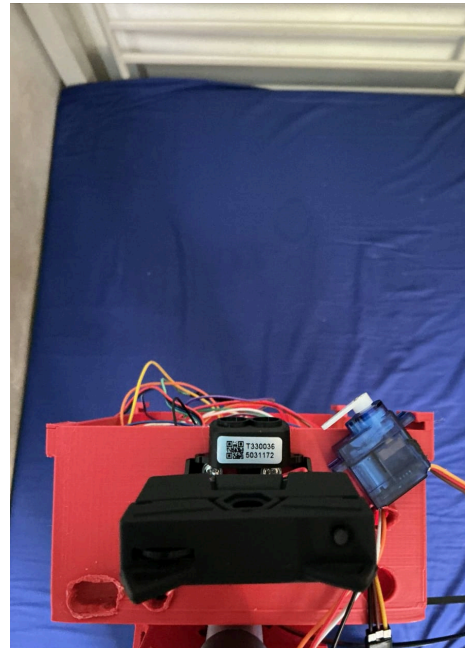
When an object enters the 30-80 cm range, the system triggers the servo to rotate to 90 degrees and immediately return to 0 degrees. This rapid movement creates a physical "nudge" or kick on the cane's handle, alerting the user to stop or change direction. We tested the system against different obstacles including a wall, a chair, and a person's leg. As shown in Table 4, the LiDAR sensor provided consistent distance readings on solid surfaces. The servo response was instantaneous once the object entered the threshold.

Table 4: Object Detection and Haptic Response Test

Object Distance	Sensor Reading (Approx.)	Servo Status	Observation
Very Close	0 cm - 29 cm	Inactive	System ignores object (Too close/Ground noise).
Critical Range	30 cm - 80 cm	Active (Nudge)	Obstacle Detected. Servo hits handle to warn user.
Safe Range	> 81 cm	Inactive	Path is clear. No feedback provided.



a) *Critical Range*



b) *Safe Range*

Fig. 8 Diagram of the detection zones.

3.4 System 4 (Human Detection System) Analysis

System 4 enhances user awareness by providing semantic feedback on human presence, distinguishing it from generic obstacle detection. The architecture interfaces a Raspberry Pi 5 with a HuskyLens AI camera via I2C, triggered by a physical push button. Upon activation, the Python firmware filters the camera data to count objects matching ID 1 ("Person"), as illustrated in Fig. 9. Once the count is complete, the Raspberry Pi 5 dynamically constructs a status message indicating the number of people detected or a clear path. We utilized the espeak-ng library to convert this text string directly into speech. This approach allows the system to work entirely offline without needing an internet connection for voice processing.

We tested the detection capability in an indoor hallway with varying numbers of people. Table 5 shows the results. The system accurately counted up to 3 people within a 2-meter range. We observed that the HuskyLens requires good lighting to accurately identify faces. In very dark conditions, the detection rate dropped, although the system still correctly reported "Path is clear" when no one was present.

Table 5: Human Detection and Audio Feedback Test

Actual Scenario	<u>HuskyLens</u> Detection Count	Audio Output Message	Accuracy
Empty Hallway	0	"Path is clear."	100%
1 Person Standing	1	"There are 1 people in front."	100%
Group of 3 People	3	"There are 3 people in front."	100%
Person Facing Away	0	"Path is clear."	-



Fig. 9 View from the HuskyLens camera showing the bounding boxes around detected faces

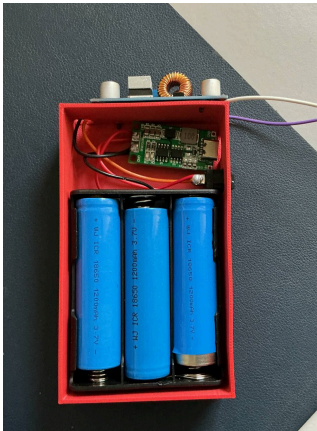
3.5 System 5 (Power Management System) Analysis

Our initial power system was a custom-made module using three rechargeable 18650 batteries, a charging board, and a voltage booster, as shown in Fig. 10(a). During testing, we found that this setup was not strong enough to handle the high power needs of the Raspberry Pi 5, especially when the camera was processing object recognition tasks. The batteries could not supply sufficient current quickly enough, which caused the system to display a low-power warning and shut down unexpectedly after only 15 to 20 minutes of operation.

To ensure the Smart Cane is reliable and safe for the user, the custom battery pack was replaced with the high-capacity 20,000 mAh commercial power bank shown in Fig. 10(b). This new unit provides a stable 3A output, which meets the power requirements of the electronics during complex tasks. Table 6 compares the performance of our initial custom design versus the final commercial solution. As a result of this upgrade, the device can operate for a significantly longer duration on a single charge, making it a practical and dependable tool for daily use without the risk of sudden shutdown.

Table 6: Power System Performance Comparison

Feature	Initial Custom Prototype	Final Commercial Solution
Battery Type	3x 18650 Li-Ion (3600mAh Total)	Lithium Polymer (20,000mAh)
Max Current Output	~1.5A (Unstable)	3.0A (Stable)
System Runtime	< 20 Minutes (Overheated)	> 8 Hours
Stability	Frequent brownouts/restarts	Continuous stable operation



(a) Initial custom-designed battery pack



(b) Final commercial power bank

Fig. 10 Evolution of the power supply.